

## **1. Executive Summary**

### **1.1 Problem & Context**

You needed a clear way to turn unstructured text into a structured Knowledge Graph (KG) while demonstrating sound LLM workflows, evidence-based reasoning, and lightweight evaluation—all within an 8-hour cap and without UI/UX overkill. We explored the concept of KGs, common build patterns from text, personality-trait modeling in graphs, and then produced a runnable, end-to-end multi-step LLM chain (entities → relations → personality traits → sanity/merge → graph build).

### **1.2 What a Knowledge Graph Is**

A KG is a graph of entities (nodes) and relations (edges) with optional properties (attributes), provenance, and confidence. It makes facts queryable (e.g., “Who works for Aster Labs?”) and auditable (linking each fact back to source text).

### **1.3 Usage & Relevance to LLMs**

KGs support LLMs by providing structured memory, grounding/evidence for answers, and reasoning over typed relationships. In this session, the KG captured not only who-did-what (relations) but also personality traits with linked evidence—useful for downstream reasoning, retrieval, or analytics.

### **1.4 Outcome**

We selected and implemented a pragmatic Multi-Step LLM Chain. To make it reproducible immediately, we shipped a ready-to-run Jupyter notebook (with an offline mock LLM), plus a Neo4j export (CSV + Cypher) so you can inspect data with a graph database.

Notebook: [multi\\_step\\_kg\\_demo.ipynb](#) (download link was provided in-session).

## **2. Objectives**

### **2.1 Primary Objective**

Create a concise, reproducible workflow that extracts entities, relations, and personality traits from raw text and assembles them into a queryable KG, prioritizing clarity, evaluation, and reasoning.

### **2.2 Secondary Objectives**

- Explain multiple viable approaches and choose the most efficient for this brief.
- Provide a practical code scaffold with validation and example queries.
- Include a Neo4j export path (CSV + Cypher) for graph inspection.
- Keep everything runnable offline while allowing easy swap-in of a real LLM.

## **3. Chosen Approach — Multi-Step LLM Chain**

### **3.1 Stages**

1. Entities: Extract typed entities with mentions (doc/sentence spans).
2. Relations: Infer typed relations (e.g., **WORKS\_FOR**, **LOCATED\_IN**) with evidence spans and confidence.
3. Traits: For **Person** entities, infer Big Five (OCEAN) traits with scores, evidence, and confidence.

4. Sanity / Alias Merge: Merge obvious aliases (e.g., “Dr. Rao” ↔ “Maya Rao”) and canonicalize identifiers.
5. Graph Build: Construct a NetworkX MultiDiGraph; traits appear as trait-nodes with scored edges from people.

### 3.2 Design Tenets

- Decomposition reduces prompt complexity and improves accuracy.
- Evidence everywhere: Each relation/trait carries a sentence-level snippet.
- Validation: Pydantic schemas ensure strict JSON; alias merge stabilizes node identity.
- Reproducibility: A single notebook with deterministic mock outputs.

## 4. Reasoning Behind the Approach

### 4.1 Why Multi-Step

- Accuracy per step: Small prompts are easier for the model to satisfy.
- Debuggability: Each stage is inspectable and re-runnable in isolation.
- Evaluation hooks: You can test entities, relations, and traits separately.

### 4.2 Alternatives Considered

- A1: Single-Pass JSON (one mega-prompt)
  - *Pros*: Fastest initial pass.
  - *Cons*: Brittle; hard to debug; limited evaluation visibility.
- A2: RAG + Function Calling (retrieval + JSON schema)
  - *Pros*: Strong grounding and scalability for long docs.
  - *Cons*: More plumbing (embeddings/store) than needed for the tight brief.

### 4.3 Why Not the Alternatives

- The single-pass approach sacrifices control and clarity; the RAG variant is excellent for long, diverse corpora but adds setup overhead not required for the present toy scope and time cap.

## 5. Implementation Details

### 5.1 Artifacts Produced

- Notebook: [multi\\_step\\_kg\\_demo.ipynb](#) (created in-session).
- Export Files (generated when you run the export cell):
  - [/mnt/data/kg\\_nodes.csv](#)
  - [/mnt/data/kg\\_edges.csv](#)
  - [/mnt/data/neo4j\\_import.cypher](#)

### 5.2 Core Data Models (Pydantic)

- Entity {name, type, aliases[], mentions[]}
- Evidence {doc\_id, sent\_id, char\_start, char\_end, text}
- Relation {head, relation, tail, evidence, confidence}
- TraitObservation {subject, trait\_id, trait\_label, score, confidence, observed\_at, method, evidence}

### 5.3 Prompt Templates (excerpts)

- Entities Prompt:  
“Task: Extract ENTITIES... Types: Person, Org, Project, Location, Date, Product... Return JSON with *name*, *type*, *aliases*, *mentions* (*doc\_id*, *sent\_id*, *char spans*). Text: *{doc\_text}*”
- Relations Prompt:  
“Task: Given the text and the entity list, extract RELATIONS... Use UPPER\_SNAKE\_CASE... Return JSON with *head*, *relation*, *tail*, *confidence*, *evidence*...”
- Traits Prompt:  
“Task: Infer personality traits for PERSON entities ONLY using OCEAN... Return JSON with *subject*, *trait\_id*, *trait\_label*, *score*, *confidence*, *observed\_at*, *method*, *evidence*...”

## 5.4 LLM Adapter & Parameters

- For immediate offline reproducibility, the notebook uses **EchoMockLLM** (deterministic JSON).
- The notebook shows how to swap in a real provider via a minimal `.chat(messages, response_format)` method (e.g., OpenAI with `temperature=0.1` and `response_format={"type":"json_object"}` for strict JSON).

## 5.5 Alias Merging

- Lightweight difflib similarity across names+aliases with a default threshold ~0.82; canonical name is the longest variant in a cluster.

## 5.6 Graph Construction

- NetworkX **MultiDiGraph** with:
  - Node attributes: *label* (type), *aliases*, *mentions*.
  - Relation edges: *kind*="RELATION", *relation*, *confidence*, *evidence*.
  - Trait edges to trait-nodes: *kind*="TRAIT", plus *score*, *confidence*, *observed\_at*, *method*, *evidence*.

## 6. Results and Outputs

### 6.1 Toy Corpus Used

- **DOC1**: “On 2 May 2024, Dr. Maya Rao joined Aster Labs as Head of Research. She previously led AI at NovaBioin Boston.”
- **DOC2**: “Ben Tan manages clinical trials at NovaBio. He coordinates weekly sprints with Aster Labs to ship reliable releases.”

### 6.2 Entity Examples (from mock extraction)

- Person: “Dr. Maya Rao” (aliases: “Maya Rao”, “Dr. Rao”, “Maya”)
- Org: “Aster Labs”, “NovaBio”
- Location: “Boston”
- Person: “Ben Tan” (alias: “Ben”)

### 6.3 Relation Examples

- (Dr. Maya Rao) —[WORKS\_FOR]→ (Aster Labs) with sentence evidence.
- (Dr. Maya Rao) —[PREVIOUSLY\_WORKED\_FOR]→ (NovaBio).
- (NovaBio) —[LOCATED\_IN]→ (Boston).

- (Ben Tan) —[WORKS\_FOR]→ (NovaBio).
- (Ben Tan) —[COLLABORATES\_WITH]→ (Aster Labs).

#### 6.4 Trait Examples (OCEAN)

- Dr. Maya Rao — Conscientiousness: 0.76 (confidence ~0.72) with evidence snippet.
- Ben Tan — Agreeableness: 0.64 (confidence ~0.60) with evidence snippet.

#### 6.5 Graph Outputs

- A merged NetworkX KG with nodes for people, orgs, locations, and trait nodes (TRAIT::OCEAN\_Conscientiousness, etc.).
- Example helper queries in the notebook:
  - people\_at(KG, "Aster Labs")
  - collaborations(KG, "Aster Labs")
  - top\_trait(KG, "OCEAN\_Conscientiousness", k=5)

#### 6.6 Neo4j Export

- The notebook writes kg\_nodes.csv and kg\_edges.csv, plus neo4j\_import.cypher that creates constraints and imports via APOC apoc.merge.node/relationship.

### 7. Validation and Evaluation

#### 7.1 Structural Validation

- Pydantic enforces JSON structure at each stage (entities, relations, traits).
- Evidence objects carry doc\_id, sent\_id, and character spans for auditability.

#### 7.2 Identity Stability

- Alias merge reduces duplicate nodes and provides a canonical identifier used across relations and traits.

#### 7.3 Functional Checks

- Graph sanity: nodes/edges load without errors; relation and trait edges carry expected keys.
- Query sanity: helper functions return expected tuples (e.g., people working at “Aster Labs”).

#### 7.4 Known Limitations / Risks

- Mock LLM produces deterministic outputs (good for demo), but not a measure of real IE accuracy.
- Without a gold-labeled subset, we did not compute precision/recall; for production, add a small hand-labeled evaluation set.
- Evidence spans are model-supplied; when swapping in a real LLM, guard with stricter JSON validation and sentence boundary checks.

### 8. Reproducibility Checklist

#### 8.1 Files

- Download and open the notebook: multi\_step\_kg\_demo.ipynb.
- (Optional) Ensure write access to /mnt/data/ for export files.

#### 8.2 Environment

- Python 3.x with: `pydantic`, `networkx`. (Both are used in the notebook; no internet required.)
- No external APIs are necessary for the demo; EchoMockLLM is built in.

### 8.3 Exact Steps

1. Open the notebook and run all cells in order.
2. Inspect printed sections: entities, relations, traits, graph nodes/edges.
3. Run helper queries: `people_at`, `collaborations`, `top_trait`.
4. Execute the Neo4j Export cell; confirm creation of:
  - `/mnt/data/kg_nodes.csv`
  - `/mnt/data/kg_edges.csv`
  - `/mnt/data/neo4j_import.cypher`
5. (Optional Neo4j) Copy the three files into Neo4j's import directory, then in Neo4j Browser:
  - `:play apoc` (ensure APOC is enabled).
  - Run the contents of `neo4j_import.cypher` to load nodes/edges.

### 8.4 Swapping in a Real LLM (optional)

- Implement a small adapter with a `.chat(messages, response_format)` method.
- Use `temperature ≈ 0.1` and JSON-mode/response-format if available.
- Replace `EchoMockLLM()` with your adapter and re-run the pipeline cells.
- For long inputs, consider chunking and (optionally) add RAG later.

### 8.5 Sharing the Session

- Commit the notebook and generated artifacts to a repo or share the `.ipynb` directly.
- Include a short README describing the steps above; optionally add screenshots of the KG queries and (if used) Neo4j Browser results.

<https://chatgpt.com/share/68f977a1-1390-800e-8847-83bce5d0e957>

The following is the ChatGPT session link utilized during my project work